



Proyecto Semana 07: API con Repository Pattern



Tu Dominio Asignado

Dominio: [El instructor te asignará tu dominio único]



IMPORTANTE: Cada aprendiz trabaja sobre un dominio diferente.



Ejemplo Genérico de Referencia

Los ejemplos usan **"Warehouse"** (Almacén) que NO está en el pool. **Debes adaptar TODO a tu dominio asignado.**

Concepto	Ejemplo Genérico	Adapta a tu Dominio
Entity	Item	{YourEntity}
Repository	IItemRepository	I{YourEntity}Repository
Fake Repo	FakeItemRepository	Fake{YourEntity}Repository



Objetivo

Implementar el **Repository Pattern** con:

- Interfaces (Protocols) para repositorios
- Implementaciones SQLAlchemy
- Fake repositories para testing
- Inyección de dependencias



Requisitos Funcionales (Adapta a tu Dominio)

Repository Interface

```
# Ejemplo genérico (Warehouse)
from typing import Protocol

class IItemRepository(Protocol):
    """Interfaz del repositorio de items"""

    async def get_by_id(self, id: int) -> Item | None: ...
    async def get_by_sku(self, sku: str) -> Item | None: ...
    async def list_all(self, skip: int, limit: int) -> list[Item]: ...
    async def create(self, item: ItemCreate) -> Item: ...
    async def update(self, id: int, data: ItemUpdate) -> Item | None: ...
    async def delete(self, id: int) -> bool: ...
    async def count(self) -> int: ...
```

SQLAlchemy Implementation

```
# Ejemplo genérico
class SQLAlchemyItemRepository:
    def __init__(self, session: AsyncSession):
        self._session = session
```

```

async def get_by_id(self, id: int) -> Item | None:
    result = await self._session.execute(
        select(ItemModel).where(ItemModel.id == id)
    )
    return result.scalar_one_or_none()

async def get_by_sku(self, sku: str) -> Item | None:
    result = await self._session.execute(
        select(ItemModel).where(ItemModel.sku == sku)
    )
    return result.scalar_one_or_none()

```

Fake Repository (Testing)

```

# Ejemplo genérico
class FakeItemRepository:
    def __init__(self):
        self._items: dict[int, Item] = {}
        self._next_id = 1

    async def get_by_id(self, id: int) -> Item | None:
        return self._items.get(id)

    async def create(self, item: ItemCreate) -> Item:
        new_item = Item(id=self._next_id, **item.model_dump())
        self._items[self._next_id] = new_item
        self._next_id += 1
        return new_item

```

Dependency Injection

```

# Ejemplo genérico
def get_item_repository(
    session: AsyncSession = Depends(get_session)
) -> IItemRepository:
    return SQLAlchemyItemRepository(session)

@router.get("/items/{id}")
async def get_item(
    id: int,
    repo: IItemRepository = Depends(get_item_repository)
):
    item = await repo.get_by_id(id)
    if not item:
        raise HTTPException(404, "Item not found")
    return item

```



Estructura del Proyecto

```

starter/
├─ main.py
├─ domain/

```

```
|   └─ entities.py
└─ repositories/
    │   └─ interfaces.py
    │   └─ sqlalchemy_repo.py
    └─ fake_repo.py
└─ services/
└─ routers/
└─ dependencies.py
└─ tests/
    └─ test_with_fake_repo.py
└─ pyproject.toml
└─ Dockerfile
└─ docker-compose.yml
```

✓ Criterios de Evaluación

Criterio	Puntos
Funcionalidad (40%)	
Interface bien definida	15
SQLAlchemy repo funciona	15
Fake repo para tests	10
Adaptación al Dominio (35%)	
Métodos específicos del negocio	12
Queries coherentes con dominio	13
Originalidad (no copia ejemplo)	10
Calidad del Código (25%)	
Inyección de dependencias	10
Tests con fake repo	10
Código limpio	5
Total	100

⚠ Política Anticopia

- ✗ **No copies** el ejemplo genérico "ItemRepository"
- ✓ **Diseña** métodos específicos de tu dominio
- ✓ **Implementa** queries relevantes para tu negocio

📚 Recursos

- [Repository Pattern](#)
- [Python Protocol](#)
- [Pool de Dominios](#)

Tiempo estimado: 3 horas

